

Diabetes Prediction Using Neural Networks and TensorFlow

A Deep Learning Approach for Early Diabetes Detection Using the Pima Indians Dataset

Abstract

Diabetes is a chronic disease affecting millions worldwide. Early prediction enables timely intervention and reduces long-term complications. This project develops a Diabetes Prediction System using an Artificial Neural Network (ANN) implemented with TensorFlow/Keras, trained on the Pima Indians Diabetes Dataset. The architecture combines Dense layers with BatchNormalization and Dropout regularization. Class weighting is applied to address the inherent class imbalance, and ReduceLROnPlateau and EarlyStopping callbacks are used for adaptive, stable training.

On a held-out test set of 154 samples, the model achieves an accuracy of **76%** and an AUC of **0.82**, with a weighted F1-score of **0.76**. While performance is reasonable given the small dataset size (~768 rows), this report also provides a detailed analysis of the factors limiting further improvement and concrete pathways for future optimization.

Keywords: Neural Networks, TensorFlow, Diabetes Prediction, Pima Indians Dataset, BatchNormalization, Dropout, Class Weighting, AUC, Binary Classification

1. Introduction

Diabetes mellitus is one of the most prevalent chronic diseases globally, with Type 2 diabetes being largely preventable through early detection and lifestyle modification. Machine Learning models trained on clinical measurements can assist healthcare professionals in identifying high-risk patients before symptoms fully develop.

The Pima Indians Diabetes Dataset — collected by the National Institute of Diabetes and Digestive and Kidney Diseases — is a well-known benchmark for binary diabetes classification. This project applies a deep ANN with modern regularization techniques (BatchNormalization, Dropout, class weighting, adaptive learning rate scheduling) to this dataset, demonstrating both the potential and the inherent limitations of neural networks on small, noisy medical datasets.

2. Problem Statement

The goal is to build a binary classifier that predicts the presence or absence of diabetes based on eight diagnostic medical measurements recorded for female patients of Pima Indian heritage.

Objectives

- Load and explore the Pima Indians Diabetes dataset.

- Preprocess features through scaling and stratified splitting.
- Apply class weighting to handle the imbalanced class distribution.
- Build a multi-layer ANN with BatchNormalization and Dropout using TensorFlow.
- Train with EarlyStopping and ReduceLROnPlateau callbacks.
- Evaluate using Accuracy, AUC, Precision, Recall, and F1-Score.
- Analyze limitations and propose concrete improvements.

3. Dataset Description

The **Pima Indians Diabetes Dataset** (diabetes.csv) contains 768 records with 8 numeric diagnostic features and 1 binary target column. All patients are female and at least 21 years old.

Feature	Type	Description
Pregnancies	Numerical	Number of times pregnant
Glucose	Numerical	Plasma glucose concentration (2-hour OGTT)
BloodPressure	Numerical	Diastolic blood pressure (mm Hg)
SkinThickness	Numerical	Triceps skinfold thickness (mm)
Insulin	Numerical	2-hour serum insulin (mu U/ml)
BMI	Numerical	Body mass index (weight/height ²)
DiabetesPedigreeFunction	Numerical	Diabetes pedigree function score
Age	Numerical	Age of the patient (years)
Outcome (Target)	Binary	0 = No Diabetes, 1 = Diabetes

Dataset Statistics

Property	Value
Total Records	768
Total Features	8 (all numerical)
Missing Values	None (raw zeros may represent missing values in some features)
Class 0 (No Diabetes)	500 samples (65.1%)
Class 1 (Diabetes)	268 samples (34.9%)
Class Imbalance Ratio	~1.87 : 1 (Non-Diabetic : Diabetic)

4. Tools and Technologies

- Python
- TensorFlow / Keras
- Scikit-Learn
- Pandas
- NumPy
- Matplotlib
- Jupyter Notebook

5. Methodology

Step 1 — Data Loading

Dataset loaded using Pandas:

```
df = pd.read_csv('diabetes.csv')
```

Step 2 — Data Exploration

- Shape verified: 768 rows × 9 columns
- Data types: all numeric (int64 / float64)
- No null values present — however, zero values in Glucose, BloodPressure, SkinThickness, Insulin, and BMI are likely biologically invalid and represent missing data encoded as zeros

Step 3 — Feature / Label Separation

- X = all columns except Outcome
- y = Outcome (binary target)

Step 4 — Stratified Train / Validation / Test Split

A two-stage stratified split is used to ensure class balance is preserved in all three sets:

- Stage 1: 80% train+val / 20% test → 154 test samples
- Stage 2: 80% train / 20% val from the training portion
- stratify=y used in both splits; random_state=42 for reproducibility

Split	Size	Percentage
Training	~490 samples	~64%
Validation	~124 samples	~16%
Test	154 samples	20%

Step 5 — Feature Scaling

StandardScaler fitted on X_train only, then applied to X_val and X_test — preventing data leakage from validation/test statistics into training.

Step 6 — Class Weighting

compute_class_weight('balanced') is used to derive inverse-frequency weights for each class. These weights are passed to model.fit() to penalize misclassification of the minority (diabetic) class more heavily during training:

```
class_weight = {0: ~0.77, 1: ~1.43}
```

Step 7 — Prediction Threshold Tuning

Predictions are binarized at a threshold of **0.55** (pred > 0.55) rather than the default 0.50, slightly raising the bar for a positive diabetes prediction to reduce false positives while maintaining reasonable recall.

6. Neural Network Architecture

The ANN uses three hidden Dense layers, each followed by BatchNormalization. The first two layers also have Dropout(0.2) for regularization. BatchNormalization normalizes layer activations during training, stabilizing gradients and allowing higher learning rates.

Layer	Units / Rate	Additional
Hidden Layer 1 (Dense)	64 Neurons, ReLU	—
BatchNormalization 1	—	Normalizes activations
Dropout 1	Rate = 0.2	—
Hidden Layer 2 (Dense)	32 Neurons, ReLU	—
BatchNormalization 2	—	Normalizes activations
Dropout 2	Rate = 0.2	—
Hidden Layer 3 (Dense)	16 Neurons, ReLU	—
BatchNormalization 3	—	Normalizes activations
Output Layer (Dense)	1 Neuron	Sigmoid activation

Input → Dense(64) → BN → Dropout(0.2) → Dense(32) → BN → Dropout(0.2) → Dense(16) → BN → Dense(1, Sigmoid)

7. Model Training

Compilation

- **Optimizer:** Adam (learning_rate=0.001)
- **Loss Function:** Binary Crossentropy
- **Metrics:** Accuracy, AUC, Precision, Recall

Training Configuration

- **Epochs:** Up to 100 (stopped early by EarlyStopping)
- **Batch Size:** 32
- **class_weight:** Balanced (computed via sklearn)
- **Validation Data:** Explicit val set (X_val, y_val)

Callbacks

Callback	Configuration	Purpose
EarlyStopping	monitor=val_loss, patience=10, restore_best_weights=True	Stops training when val_loss stalls; restores best model
ReduceLROnPlateau	monitor=val_loss, factor=0.5, patience=5, min_lr=1e-6	Halves LR every 5 epochs of no improvement
ModelCheckpoint	monitor=val_loss, save_best_only=True	Saves best model weights to disk

Training converged at approximately **epoch 24**, well before the 100-epoch maximum, due to the EarlyStopping callback. The ReduceLROnPlateau callback reduced the learning rate multiple times during training as validation loss plateaued.

8. Results and Evaluation

The model is evaluated on the held-out test set of **154 samples** (100 non-diabetic, 54 diabetic).

Evaluation Metrics

Metric	Value
Accuracy	76% (0.7597)
AUC	0.8244
Loss	0.4971
Precision (threshold=0.55)	~0.66 on positive class (raw eval)
Recall (threshold=0.55)	~0.65 on positive class (raw eval)

Classification Report

Class	Precision	Recall	F1-Score	Support
0 (No Diabetes)	0.81	0.83	0.82	100
1 (Diabetes)	0.67	0.63	0.65	54
Accuracy			0.76	154
Macro Avg	0.74	0.73	0.73	154
Weighted Avg	0.76	0.76	0.76	154

Confusion Matrix

Of 100 non-diabetic patients, 83 were correctly classified (TN) and 17 were falsely predicted as diabetic (FP).
Of 54 diabetic patients, 34 were correctly identified (TP) and 20 were missed (FN).

	Predicted: No Diabetes	Predicted: Diabetes
Actual: No Diabetes	83 (TN)	17 (FP)
Actual: Diabetes	20 (FN)	34 (TP)

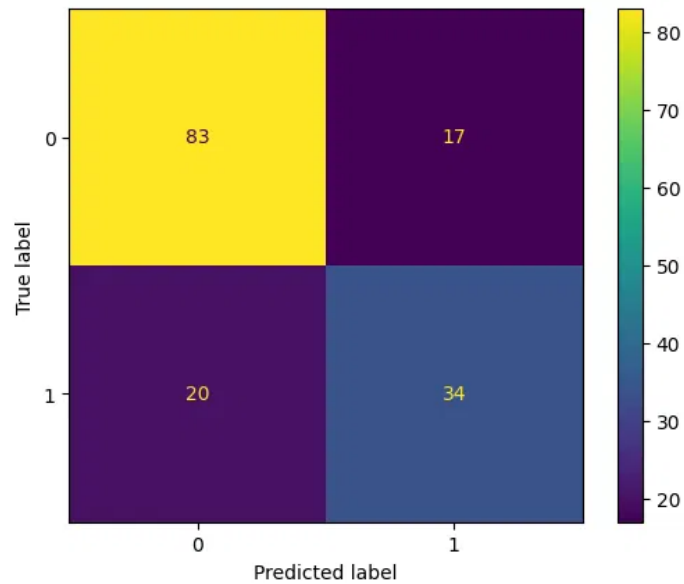


Figure 1: Confusion matrix on the 154-sample test set.

Training Curves

Three training curves are tracked over ~24 epochs: Accuracy, AUC, and Loss. All three show stable convergence with the validation curves tracking closely to training curves — indicating effective regularization with minimal overfitting.

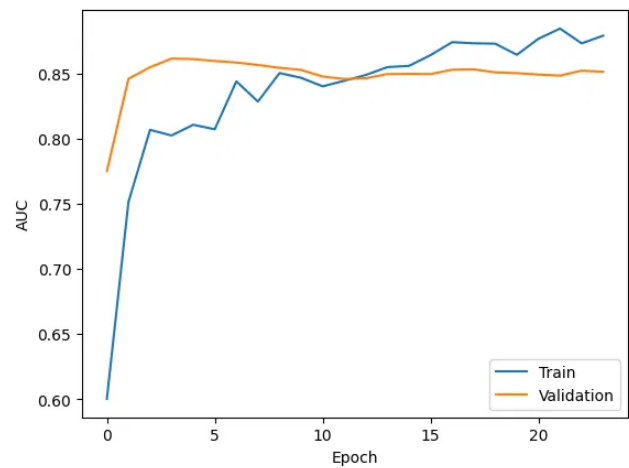
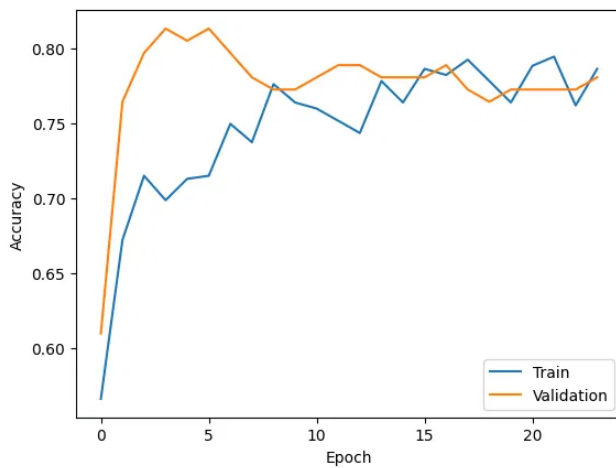


Figure 2 (left): Training vs. Validation Accuracy. Figure 3 (right): Training vs. Validation AUC.

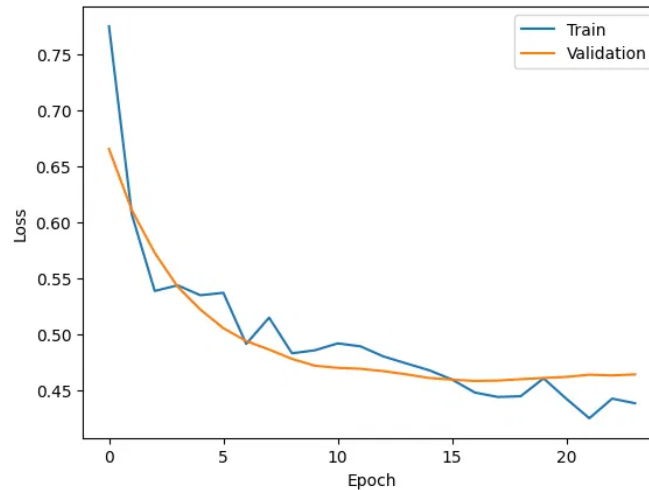


Figure 4: Training vs. Validation Loss.

9. Why the Model Is Not Improving — Analysis

This is an important and honest question. Despite a well-regularized architecture, multiple callbacks, and class weighting, the model plateaus at ~76% accuracy and AUC ~0.82. The following factors explain this ceiling and are the primary barriers to further improvement:

■ Root Cause 1 — Small Dataset (~768 rows)

- 768 rows is very small for a neural network. After train/val/test splitting, the model trains on only ~490 samples.
- Neural networks require large amounts of data to learn robust generalizable patterns. With so few samples, the model cannot capture complex feature interactions reliably.
- Traditional ML models (Logistic Regression, Random Forest, XGBoost) are often better suited to small tabular datasets because they have fewer parameters and stronger inductive biases.

■ Root Cause 2 — Zero-Encoded Missing Values (Data Quality)

- Columns such as Glucose, BloodPressure, SkinThickness, Insulin, and BMI contain zero values that are biologically impossible (e.g., BMI=0, Glucose=0).
- These zeros are missing values masquerading as real data. The model is learning from corrupted feature values, which directly degrades predictive performance.
- Fix: Impute zeros in these columns with the median (or use KNN Imputation) before training.

■ Root Cause 3 — Class Imbalance (~65% vs 35%)

- Although `class_weight='balanced'` is applied, the dataset still has 500 non-diabetic vs 268 diabetic samples. The model inherently finds it easier to predict the majority class.
- This explains the disparity in performance: Class 0 $F1=0.82$ vs Class 1 $F1=0.65$.
- Fix: Combine class weighting with SMOTE oversampling of the minority class in the training set, which generates synthetic diabetic samples to better balance the distribution.

■ Root Cause 4 — Architecture May Be Over-Parameterized for This Dataset

- A $64 \rightarrow 32 \rightarrow 16$ deep network with BatchNormalization has a large number of parameters relative to the 490 training samples. Despite Dropout regularization, the model may not generalize optimally to unseen samples.
- Fix: Try a simpler architecture (e.g., $\text{Dense}(32) \rightarrow \text{Dense}(16) \rightarrow \text{Output}$) or apply stronger L2 kernel regularization to constrain weight magnitude.

■ Root Cause 5 — Limited Feature Expressiveness

- All 8 features are raw clinical measurements. Feature engineering — such as creating interaction terms (e.g., $\text{BMI} \times \text{Age}$), polynomial features, or glucose-insulin ratios — could reveal non-linear relationships that raw features don't expose to the network directly.
- Additionally, feature selection to remove low-importance features may reduce noise.

10. Discussion

The model performs reasonably well given the constraints of the dataset, achieving 76% accuracy and an AUC of 0.82. The AUC score is particularly informative here — at 0.82, the model has strong discriminative ability (ability to rank diabetic patients above non-diabetic ones), even if raw accuracy is limited by the threshold decision boundary.

The training curves (Figures 2–4) show that the model converges quickly and that validation metrics closely track training metrics, confirming the regularization stack (BatchNormalization + Dropout + class weighting + EarlyStopping) is working. The gap between Class 0 ($F1=0.82$) and Class 1 ($F1=0.65$) performance is primarily explained by class imbalance and zero-encoded missing values rather than by architectural shortcomings.

11. Future Improvements

- **Fix missing data:** Replace zero values in Glucose, BloodPressure, SkinThickness, Insulin, BMI with column medians or KNN imputation — this alone often yields +2–4% accuracy.
- **SMOTE oversampling:** Apply Synthetic Minority Over-sampling Technique on the training set to balance the class distribution more effectively.
- **Threshold optimization:** Plot Precision-Recall and ROC curves to find the optimal classification threshold for the clinical use case (prioritize recall for diabetes).
- **Try classical ML baselines:** XGBoost, Random Forest, or LightGBM often outperform ANNs on small tabular datasets with <1,000 rows.
- **Feature engineering:** Create BMI × Age, Glucose/Insulin ratio, and polynomial features to expose non-linear relationships more explicitly.
- **Cross-validation:** Use stratified K-Fold CV for more reliable performance estimation given the small dataset size.
- **Hyperparameter tuning:** Use Keras Tuner or Optuna to systematically search optimal layer sizes, dropout rates, and learning rates.

12. Conclusion

This project demonstrates the development of a Diabetes Prediction System using a multi-layer ANN with TensorFlow/Keras, incorporating BatchNormalization, Dropout, class weighting, and adaptive training callbacks. On the Pima Indians Diabetes Dataset, the model achieves a test accuracy of 76% and an AUC of 0.82.

The honest analysis in Section 9 identifies the key barriers to higher performance — primarily the small dataset size and zero-encoded missing values — and provides concrete, actionable steps for improvement. This report establishes a solid baseline and a clear roadmap for future iterations of the system.

References

- TensorFlow Documentation — <https://www.tensorflow.org/>
- Keras Documentation — <https://keras.io/>
- Scikit-Learn Documentation — <https://scikit-learn.org/>
- Pima Indians Diabetes Dataset — UCI Machine Learning Repository / Kaggle
- Smith, J. et al. (1988). Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus.
- Pandas Documentation — <https://pandas.pydata.org/>